Practical 3: Implementation and complexity analysis of algorithms involving divide and conquer strategy

Quicksort Algorithm

Objective

- The primary objective of Quicksort algoritm is to efficiently sort an arry or a list of elements using divide and conquer strategy.
- To efficiently sort the list or array without requiring additional storage for merginge(unlike mergesort)
- To achieve an average time complexity of O(n logn) while minimizing space overhead.

Theory

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array

There are mainly three steps in the algorithm:

- Choose a Pivot: Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
- Partition the Array: Re arrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right.
- Recursively Call: Recursively apply the same process to the two partitioned sub-arrays. Base Case: The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Algorithm

Algorithm 1 Quick Sort

procedure PARTITION($A, low, high$)
  $pivot \leftarrow A[low]$
  $x \leftarrow low + 1$
  $y \leftarrow high$
  while $x \leq y$ **do**
    while $x \leq high$ **and** $A[x] \leq pivot$ **do**
      $x \leftarrow x + 1$
    end while
    while $y \geq low + 1$ **and** $A[y] \geq pivot$ **do**
      $y \leftarrow y - 1$
    end while
    if $x < y$ **then**
      Swap($A[x], A[y]$)
    end if
  end while
  Swap($A[low], A[y]$)
  return y
end procedure
procedure QUICKSORT($A, low, high$)
  if $low < high$ **then**
    $p \leftarrow$ PARTITION($A, low, high$)
    QUICKSORT($A, low, p - 1$)
    QUICKSORT($A, p + 1, high$)
  end if
end procedure

Python Code

```python
def Partition(A,low,high):
    pivot=A[low]
    x=low+1
    y=high

    while x<=y:
        while x<=high and A[x]<=pivot :
            x=x+1
        while y>=(low+1) and A[y]>=pivot:
            y=y-1
        if x<y:
            A[x],A[y]=A[y],A[x]
    A[low],A[y]=A[y],A[low]
    return y

def quickSort(A,low,high):
    if low<high:
        p=Partition(A,low,high)
        quickSort(A,low,p-1)
        quickSort(A,p+1,high)
```

Output

```
arr=[11,44,63,23,45,667,223,445,89,42,222]
print('Original array: ', arr)
quickSort(arr,0,len(arr)-1)
```

```
print('sorted array: ', arr)

Original array:  [11, 44, 63, 23, 45, 667, 223, 445, 89, 42, 222]
sorted array:  [11, 23, 42, 44, 45, 63, 89, 222, 223, 445, 667]
```

Complexity Analysis

Time Complexity:

- BestCase/AvegrageCase: $O(n \log n)$
- WorstCase: $O(n^2)$

Space Complexity:

- BestCase/AvegrageCase: $O(\log n)$
- WorstCase: $O(n)$

HeapSort Algorithm

Objective

- The primary objective of HeapSort algoritm is to consistently sort data in $O(n \log n)$ for best,average,worst time complexity.
- To be highly memory efficient because it only need $O(1)$ auxillary space.
- To provide a stable upper bound on execution time, which is critical for real-time systems.

Theory

Heap Sort is a comparison-based sorting algorithm based on the Binary Heap data structure.The features of HeapSort are:

- It is an optimized version of selection sort.
- The algorithm repeatedly finds the maximum (or minimum) element and swaps it with the last (or first) element.
- Using a binary heap allows efficient access to the max (or min) element in O(log n) time instead of $O(n)$.
- Overall, Heap Sort achieves a time complexity of $O(n \log n)$. Base Case: The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Algorithm 2 Heap Sort

procedure MaxHeapify(A, n, i)
  $largest \leftarrow i$
  $l \leftarrow 2 \times i$
  $r \leftarrow 2 \times i + 1$
  if $l \leq n$ **and** $A[l] > A[largest]$ **then**
    $largest \leftarrow l$
  end if
  if $r \leq n$ **and** $A[r] > A[largest]$ **then**
    $largest \leftarrow r$
  end if
  if $largest \neq i$ **then**
    Swap($A[i], A[largest]$)
    MaxHeapify($A, n, largest$)
  end if
end procedure
procedure HeapSort(A, n)
  for $i \leftarrow \lfloor n/2 \rfloor$ **to** 1 **step** -1 **do**
    MaxHeapify(A, n, i)
  end for
  for $i \leftarrow n$ **to** 1 **step** -1 **do**
    Swap($A[1], A[i]$)
    MaxHeapify($A, i-1, 1$)
  end for
  return $A[1:n]$
end procedure

Python Code

```python
def MaxHeapify(A,n,i):
    largest=i
    l=2*i
    r=(2*i)+1

    if (l<=n and A[l]>A[largest]):
        largest=l
    if (r<=n and A[r]>A[largest]):
        largest=r

    if largest!=i:
        A[i], A[largest] = A[largest], A[i]
        MaxHeapify(A,n,largest)

def HeapSort(A,n):
    for i in range(n//2,0,-1):
        MaxHeapify(A,n,i)
    for i in range(n,0,-1):
        A[1],A[i]=A[i],A[1]
        MaxHeapify(A,i-1,1)
    return A[1:]
A = [None,4, 10, 3, 5, 1] # using dummy index
sorted_A = HeapSort(A, len(A) - 1)
print(sorted_A)
```

Output

```
A = [None,4, 10, 3, 5, 1] # using dummy index
sorted_A = HeapSort(A, len(A) - 1)
```

```
print("The sorted array is : ",sorted_A)

The sorted array is : [1, 3, 4, 5, 10]
```

Complexity Analysis

Time Complexity:

- BestCase/AvegrageCase/WorstCase: $O(n \log n)$

Space Complexity:

- Auxillary heap construction: $O(1)$
- Recursive heapify: $O(\log n)$

Discussion

Quick Sort and Heap Sort were implemented and analyzed. Quick Sort uses the divide-and-conquer technique and provides an average time complexity of $O(n \log n)$, but its worst-case time complexity is $O(n^2)$. Heap Sort builds a max heap and guarantees $O(n \log n)$ time complexity in all cases while requiring only constant auxiliary space.

Conclusion

Both Quick Sort and Heap Sort are efficient sorting algorithms. Quick Sort is generally faster in the average case, whereas Heap Sort provides consistent $O(n \log n)$ performance with lower memory usage. The choice of algorithm depends on the application's performance and memory requirements.